

</>

The Anuj DECODED

</>

```
01 #include <iostream>
02 #include <vector>
03 #include <unordered_map>
04 using namespace std;
05
06 class Solution {
07 public:
08     vector<int> twoSum(
09         vector<int>& nums,
10         int target) {
11         unordered_map<int, int> mp;
12         for (int i = 0; i < nums.size(); ++i) {
13             int diff = target - nums[i];
14             if (mp.count(diff)) {
15                 return {mp[diff], i};
16             }
17             mp[nums[i]] = i;
18         }
19         return {};
20     }
};
```



```
01 // Think Deeper
02 // Write Cleaner
03 // Solve Smarter
04 // Get Stronger
05 // Every day
06 // a better
07 // problem solver
```

CODE. SOLVE. **IMPROVE**. REPEAT.



THINK
DEEPER



WRITE
CLEANER



SOLVE
SMARTER



GET
STRONGER

“

Problem today,
Stronger tomorrow.

This Notebook Belongs To:

ANUJ

1477 · Find Two Non-overlapping Sub-arrays Each With Target Sum

LeetCode · Daily Question

Medium

Sliding Window

DP

1. What Type of Question Is This?

This is a **sliding window + running-best DP** problem. Because every element is a **positive** integer, the sum of a window only ever grows as you extend it right and only ever shrinks as you contract it from the left — that monotonic behavior is exactly what makes a classic two-pointer sliding window valid here (unlike arrays with negative numbers, where sliding windows don't work). The "find the best pairing of two non-overlapping valid pieces" structure is a common pattern: find all valid pieces efficiently, then combine each one with the best compatible piece found before it.

2. What Is Given?

- An array of **positive** integers, **arr**.
- An integer **target**.
- You must find **two non-overlapping** (no shared indices) sub-arrays that each sum to exactly **target**.

3. What Do We Need to Find?

Return the **minimum possible sum of the two sub-arrays' lengths** — among every valid pair of non-overlapping target-sum sub-arrays, the one where the combined length is smallest. Return **-1** if no such pair exists.

Function Signature

■ C++

```
class Solution {
public:
    int minSumOfLengths(vector<int>& arr, int target) {

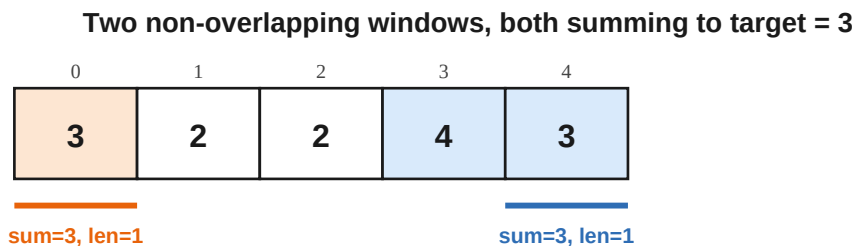
    }
};
```

Examples Walkthrough

Example 1 — used as the running example for every code sample below

```
Input: arr = [3,2,2,4,3], target = 3
Output: 2
```

Only two sub-arrays sum to 3: [3] at index 0 (length 1) and [3] at index 4 (length 1). They don't overlap, so their combined length is $1 + 1 = 2$.



minLen[] remembers the best window found so far to the LEFT
so pairing with a later window is an $O(1)$ lookup, not a re-scan.

Figure 1 — the two valid single-element windows for Example 1, at opposite ends of the array.

Example 2 — multiple candidates, pick the best pairing

```
Input: arr = [7,3,4,7], target = 7
Output: 2
```

There are three non-overlapping sub-arrays summing to 7: [7] (index 0), [3,4] (indices 1-2), and [7] (index 3) — but we only need to *choose two* of them that don't overlap. Picking the first and third (both length 1) gives the smallest combined length: $1 + 1 = 2$.

Example 3 — only one valid sub-array exists

```
Input: arr = [4,3,2,6,2,3,4], target = 6
Output: -1
```

Only one sub-array sums to 6. Since a *second*, non-overlapping one is required and none exists, the answer is **-1**.

Constraints

- $1 \leq \text{arr.length} \leq 10^5$
- $1 \leq \text{arr}[i] \leq 1000$
- $1 \leq \text{target} \leq 10^8$

n up to 100,000 rules out anything $O(n^2)$ — the intended solution needs $O(n)$ or $O(n \log n)$, which is exactly what the sliding-window technique (valid here because every element is positive) provides.

Overall Workflow & How to Think About It

Step-by-step reasoning to arrive at the right approach:

- **Step 1 — Find every valid sub-array efficiently.** Since all elements are positive, a two-pointer sliding window (right pointer extends, left pointer shrinks whenever the sum exceeds target) finds, for each right endpoint, the unique window ending there that could possibly sum to target — in $O(n)$ total across the whole array, since each pointer only moves forward.
- **Step 2 — Don't just find valid windows — remember the best one seen so far.** As you scan left to right, maintain $\text{minLen}[i]$ = the length of the **shortest** valid (sum == target) sub-array found anywhere within $\text{arr}[0..i]$. This is a simple running minimum, updated at every position.
- **Step 3 — Whenever a new valid window is found, try pairing it with the best earlier one.** If the current window spans $[\text{left}, \text{right}]$, anything it could pair with non-overlapping must be entirely within $\text{arr}[0..\text{left}-1]$ — exactly what $\text{minLen}[\text{left}-1]$ already tells you, in $O(1)$. Combine: candidate answer = $\text{minLen}[\text{left}-1] + (\text{right} - \text{left} + 1)$.
- **Step 4 — Track the overall best candidate** across every valid window found during the scan, and return it (or -1 if no valid pairing was ever found).

Algorithm Flow

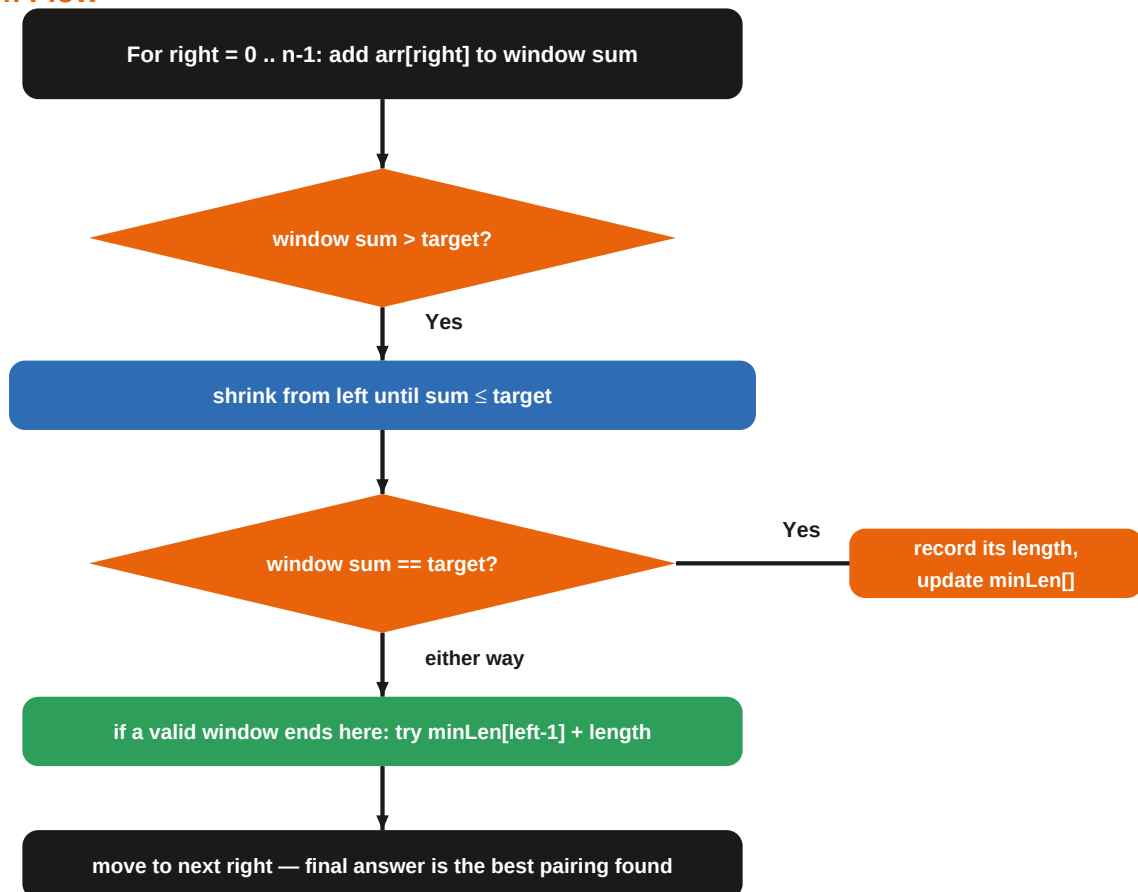


Figure 2 — the sliding window and the prefix-best array work together in a single left-to-right pass.

Which Algorithm Is Best?

A single sliding-window pass, maintaining the running prefix-minimum length array alongside it, finds and pairs every valid window in one $O(n)$ sweep. The brute-force and "better" tiers below build up to this: first finding all valid sub-arrays exhaustively and pairing them all, then finding sub-arrays efficiently but still re-deriving the best earlier pairing naively, before finally folding both steps together into the single linear pass.

Approach 1 — Brute Force (Find All Valid Sub-arrays, Try Every Pair)

Idea: Directly translate the problem statement: find every sub-array that sums to target by checking all (start, end) ranges, then try every possible pair of them, keeping the smallest combined length among pairs that don't overlap.

Steps

- For every start index i , extend a running sum rightward; record (i, j, length) whenever the running sum equals target, and stop extending once it exceeds target (since all elements are positive, it can only grow further).
- Collect every such valid sub-array found.
- For every ordered pair of found sub-arrays, check whether the first ends before the second starts.
- Track the minimum combined length among all valid (non-overlapping) pairs.
- Return that minimum, or -1 if no valid pair was found.

Approach 1 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minSumOfLengths(vector<int>& arr, int target) {
        int n = arr.size();
        vector<array<int,3>> subs;    // {start, end, length}

        // Find every sub-array summing to target, by brute force.
        for (int i = 0; i < n; i++) {
            long long sum = 0;
            for (int j = i; j < n; j++) {
                sum += arr[j];
                if (sum == target) subs.push_back({i, j, j - i + 1});
                if (sum > target) break;    // positive numbers only grow the sum further
            }
        }

        int best = INT_MAX;
        // Try every PAIR of found sub-arrays, keep the best non-overlapping pairing.
        for (int a = 0; a < (int)subs.size(); a++) {
            for (int b = 0; b < (int)subs.size(); b++) {
                if (a == b) continue;
                if (subs[a][1] < subs[b][0]) {    // subs[a] ends before subs[b] starts
                    best = min(best, subs[a][2] + subs[b][2]);
                }
            }
        }
        return best == INT_MAX ? -1 : best;
    }
};

int main() {
    vector<int> arr = {3,2,2,4,3};
    int target = 3;
    Solution sol;
    cout << sol.minSumOfLengths(arr, target) << endl;    // Expected output: 2
    return 0;
}
```

■ Python

```
class Solution:
    def minSumOfLengths(self, arr, target):
```

```

n = len(arr)
subs = [] # (start, end, length)

# Find every sub-array summing to target, by brute force.
for i in range(n):
    s = 0
    for j in range(i, n):
        s += arr[j]
        if s == target:
            subs.append((i, j, j - i + 1))
        if s > target:
            break # positive numbers only grow the sum further

best = float("inf")
# Try every PAIR of found sub-arrays, keep the best non-overlapping pairing.
for a in range(len(subs)):
    for b in range(len(subs)):
        if a == b:
            continue
        if subs[a][1] < subs[b][0]: # subs[a] ends before subs[b] starts
            best = min(best, subs[a][2] + subs[b][2])

return -1 if best == float("inf") else best

if __name__ == "__main__":
    arr, target = [3,2,2,4,3], 3
    print(Solution().minSumOfLengths(arr, target)) # Expected output: 2

```

■ JavaScript

```

class Solution {
    minSumOfLengths(arr, target) {
        const n = arr.length;
        const subs = []; // [start, end, length]

        // Find every sub-array summing to target, by brute force.
        for (let i = 0; i < n; i++) {
            let s = 0;
            for (let j = i; j < n; j++) {
                s += arr[j];
                if (s === target) subs.push([i, j, j - i + 1]);
                if (s > target) break; // positive numbers only grow the sum further
            }
        }

        let best = Infinity;
        // Try every PAIR of found sub-arrays, keep the best non-overlapping pairing.
        for (let a = 0; a < subs.length; a++) {
            for (let b = 0; b < subs.length; b++) {
                if (a === b) continue;
                if (subs[a][1] < subs[b][0]) { // subs[a] ends before subs[b] starts
                    best = Math.min(best, subs[a][2] + subs[b][2]);
                }
            }
        }
        return best === Infinity ? -1 : best;
    }
}

const arr = [3,2,2,4,3], target = 3;
console.log(new Solution().minSumOfLengths(arr, target)); // Expected output: 2

```

Complexity: Finding all valid sub-arrays costs up to $O(n^2)$ in the worst case (many short valid windows). Pairing all of them costs $O(m^2)$ where m is the number of valid sub-arrays found, which can itself be $O(n)$ — giving up to $O(n^2)$ overall. With n up to 100,000, this is far too slow.

Approach 2 — Better (Binary Search for Windows, but Re-scan Earlier Ones Each Time)

Idea: Apply the Step 1 insight to find all valid windows efficiently (binary search on prefix sums, since they're strictly increasing), but for pairing, still look back through every previously found window each time instead of maintaining a running best.

Steps

- Build the prefix-sum array once, in $O(n)$.
- For each end index, binary search the prefix-sum array for the start index that makes the window sum exactly target — $O(\log n)$ per end index.
- Collect all (start, end) windows found this way.
- For each window (in order found), scan every earlier window to find the shortest one that ends before the current one starts.
- Track the best combined length across all such compatible pairs.

Approach 2 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minSumOfLengths(vector<int>& arr, int target) {
        int n = arr.size();
        vector<int> prefix(n + 1, 0);
        for (int i = 0; i < n; i++) prefix[i+1] = prefix[i] + arr[i];

        // For each end index, binary search for the start that makes the sum == target
        // (valid because prefix sums are strictly increasing – all elements are positive).
        vector<array<int,2>> windows; // {start, end} of each valid sub-array
        for (int end = 0; end < n; end++) {
            int want = prefix[end+1] - target;
            int idx = lower_bound(prefix.begin(), prefix.begin() + end + 1, want) - prefix.begin();
            if (idx <= end && prefix[idx] == want) windows.push_back({idx, end});
        }

        int best = INT_MAX;
        // For each valid window, re-scan all EARLIER windows for the best compatible one.
        for (int a = 0; a < (int)windows.size(); a++) {
            int bestBefore = INT_MAX;
            for (int b = 0; b < a; b++) {
                if (windows[b][1] < windows[a][0]) {
                    bestBefore = min(bestBefore, windows[b][1] - windows[b][0] + 1);
                }
            }
            if (bestBefore != INT_MAX) {
                best = min(best, bestBefore + (windows[a][1] - windows[a][0] + 1));
            }
        }
        return best == INT_MAX ? -1 : best;
    }
};

int main() {
    vector<int> arr = {3,2,2,4,3};
    int target = 3;
    Solution sol;
    cout << sol.minSumOfLengths(arr, target) << endl; // Expected output: 2
    return 0;
}
```

Python

```
import bisect

class Solution:
    def minSumOfLengths(self, arr, target):
        n = len(arr)
        prefix = [0] * (n + 1)
        for i in range(n):
            prefix[i+1] = prefix[i] + arr[i]

        # For each end index, binary search for the start that makes the sum == target
        # (valid because prefix sums are strictly increasing – all elements are positive).
        windows = [] # (start, end) of each valid sub-array
        for end in range(n):
            want = prefix[end+1] - target
            idx = bisect.bisect_left(prefix, want, 0, end + 1)
            if idx <= end and prefix[idx] == want:
                windows.append((idx, end))

        best = float("inf")
        # For each valid window, re-scan all EARLIER windows for the best compatible one.
        for a in range(len(windows)):
            bestBefore = float("inf")
            for b in range(a):
                if windows[b][1] < windows[a][0]:
                    bestBefore = min(bestBefore, windows[b][1] - windows[b][0] + 1)
            if bestBefore != float("inf"):
                best = min(best, bestBefore + (windows[a][1] - windows[a][0] + 1))

        return -1 if best == float("inf") else best

if __name__ == "__main__":
    arr, target = [3,2,2,4,3], 3
    print(Solution().minSumOfLengths(arr, target)) # Expected output: 2
```

JavaScript

```
class Solution {
    minSumOfLengths(arr, target) {
        const n = arr.length;
        const prefix = new Array(n + 1).fill(0);
        for (let i = 0; i < n; i++) prefix[i+1] = prefix[i] + arr[i];

        // For each end index, binary search for the start that makes the sum == target
        // (valid because prefix sums are strictly increasing – all elements are positive).
        const lowerBound = (want, hi) => {
            let lo = 0, h = hi;
            while (lo < h) {
                const mid = (lo + h) >> 1;
                if (prefix[mid] < want) lo = mid + 1; else h = mid;
            }
            return lo;
        };

        const windows = []; // [start, end] of each valid sub-array
        for (let end = 0; end < n; end++) {
            const want = prefix[end+1] - target;
            const idx = lowerBound(want, end + 1);
            if (idx <= end && prefix[idx] === want) windows.push([idx, end]);
        }

        let best = Infinity;
        // For each valid window, re-scan all EARLIER windows for the best compatible one.
        for (let a = 0; a < windows.length; a++) {
            let bestBefore = Infinity;
            for (let b = 0; b < a; b++) {
                if (windows[b][1] < windows[a][0]) {
                    bestBefore = Math.min(bestBefore, windows[b][1] - windows[b][0] + 1);
                }
            }
            if (bestBefore !== Infinity) {
                best = Math.min(best, bestBefore + (windows[a][1] - windows[a][0] + 1));
            }
        }

        return best === Infinity ? -1 : best;
    }
}
```

```

        best = Math.min(best, bestBefore + (windows[a][1] - windows[a][0] + 1));
    }
    return best === Infinity ? -1 : best;
}

const arr = [3,2,2,4,3], target = 3;
console.log(new Solution().minSumOfLengths(arr, target)); // Expected output: 2

```

Complexity: Finding all windows costs $O(n \log n)$ thanks to binary search. But pairing them still costs $O(m^2)$ in the worst case (m = number of valid windows, up to $O(n)$) — $O(n^2)$ overall in the worst case, better in practice than Approach 1 but not safely fast enough for the full $n = 100,000$ constraint.

Approach 3 — Optimal (Sliding Window + Running Prefix-Minimum, One Pass)

Idea: Fold window-finding and best-pairing into a single left-to-right sweep. The two-pointer sliding window finds each valid window in amortized $O(1)$ per step (both pointers only ever move forward), and the running `minLen[]` array (Step 2–3 of the workflow) gives the best compatible earlier window in $O(1)$ as well — no re-scanning, no binary search needed at all.

Steps

- Maintain a sliding window `[left, right]` and its running sum; extend right, and shrink from the left whenever the sum exceeds target.
- Whenever the window sum equals target exactly, that's a valid window of length `right - left + 1`.
- If `minLen[left-1]` is already known (some valid window exists entirely before this one), immediately try `minLen[left-1] + current length` as a candidate answer.
- Update `minLen[right]` as the minimum of `minLen[right-1]` and this window's length (or leave it as `minLen[right-1]` if no valid window ends exactly at right).
- After the scan, the tracked best candidate is the answer (or -1 if none was ever found).

Approach 3 — Code (all languages run Example 1)

■ C++

```
#include <bits/stdc++.h>
using namespace std;

class Solution {
public:
    int minSumOfLengths(vector<int>& arr, int target) {
        int n = arr.size();
        vector<int> minLen(n, INT_MAX); // minLen[i]: shortest valid window fully within arr[0..i]

        int left = 0;
        long long sum = 0;
        int ans = INT_MAX;

        for (int right = 0; right < n; right++) {
            sum += arr[right];
            while (sum > target) {
                sum -= arr[left];
                left++;
            }

            int curBest = INT_MAX;
            if (sum == target) {
                int curLen = right - left + 1;
                curBest = curLen;
                if (left - 1 >= 0 && minLen[left - 1] != INT_MAX) {
                    ans = min(ans, minLen[left - 1] + curLen);
                }
            }
            int prior = (right - 1 >= 0) ? minLen[right - 1] : INT_MAX;
            minLen[right] = min(prior, curBest);
        }
        return ans == INT_MAX ? -1 : ans;
    }
};

int main() {
    vector<int> arr = {3,2,2,4,3};
    int target = 3;
    Solution sol;
    cout << sol.minSumOfLengths(arr, target) << endl; // Expected output: 2
}
```

```

    return 0;
}

```

■ Python

```

class Solution:
    def minSumOfLengths(self, arr, target):
        n = len(arr)
        INF = float("inf")
        minLen = [INF] * n # minLen[i]: shortest valid window fully within arr[0..i]

        left = 0
        s = 0
        ans = INF

        for right in range(n):
            s += arr[right]
            while s > target:
                s -= arr[left]
                left += 1

            curBest = INF
            if s == target:
                curLen = right - left + 1
                curBest = curLen
                if left - 1 >= 0 and minLen[left - 1] != INF:
                    ans = min(ans, minLen[left - 1] + curLen)

            prior = minLen[right - 1] if right - 1 >= 0 else INF
            minLen[right] = min(prior, curBest)

        return -1 if ans == INF else ans

if __name__ == "__main__":
    arr, target = [3,2,2,4,3], 3
    print(Solution().minSumOfLengths(arr, target)) # Expected output: 2

```

■ JavaScript

```

class Solution {
    minSumOfLengths(arr, target) {
        const n = arr.length;
        const INF = Infinity;
        const minLen = new Array(n).fill(INF); // shortest valid window fully within arr[0..i]

        let left = 0;
        let sum = 0;
        let ans = INF;

        for (let right = 0; right < n; right++) {
            sum += arr[right];
            while (sum > target) {
                sum -= arr[left];
                left++;
            }

            let curBest = INF;
            if (sum === target) {
                const curLen = right - left + 1;
                curBest = curLen;
                if (left - 1 >= 0 && minLen[left - 1] !== INF) {
                    ans = Math.min(ans, minLen[left - 1] + curLen);
                }
            }
            const prior = right - 1 >= 0 ? minLen[right - 1] : INF;
            minLen[right] = Math.min(prior, curBest);
        }
        return ans === INF ? -1 : ans;
    }
}

const arr = [3,2,2,4,3], target = 3;

```

```
console.log(new Solution().minSumOfLengths(arr, target)); // Expected output: 2
```

Complexity: $O(n)$ time: the sliding window's left and right pointers each move at most n steps total (amortized $O(1)$ per iteration of the outer loop), and every other operation per step is $O(1)$. $O(n)$ space for the `minLen[]` array. This comfortably handles the full $n = 100,000$ constraint with room to spare.

Side-by-Side Comparison

1. Brute Force	Find all valid windows by brute scan, try every pair	$O(n^2)$ — too slow at $n=10^5$	$O(n^2)$ for found windows
2. Better	Binary search finds windows in $O(n \log n)$, but pairing re-scans each window case for pairing	$O(n^2 \log n)$	$O(n)$ for found windows
3. Optimal	Sliding window + running prefix-min array, single $O(n)$ pass	$O(n)$	$O(n)$ for the minLen[] array

Key Takeaways to Remember

- **All-positive array + fixed target sum** is the classic signal for a sliding window: the window sum only moves in one direction as each pointer moves, so both pointers can advance monotonically without ever needing to backtrack.
- **"Find the best pairing of two non-overlapping valid pieces"** almost always wants a running *prefix-best* array (here, minLen[]) maintained alongside the main scan — it turns "what's the best compatible piece before this one" into an $O(1)$ lookup instead of a re-scan.
- A crucial correctness detail: the prefix-best array must only ever be looked up at **left-1** (strictly before the current window starts), never at the window's own position or anywhere that could overlap it — get this index wrong and the answer silently allows overlapping pairs.
- Binary search on prefix sums (Approach 2) is a fine way to *find* valid windows when direct two-pointer reasoning isn't obvious yet, but it doesn't by itself solve the pairing step — that still needs its own running-best structure.
- All three tiers were validated against **Example 1** (arr = [3,2,2,4,3], target = 3 → 2) across C++, Python, and JavaScript, checked against Examples 2 and 3 as well, and cross-verified against brute force over 2000 randomized cases.